

Towards Privacy-Preserving Unmanned Aerial Vehicles Shared Logistics via Dynamic Sanitizable Signature with Multiple Sanitizers

Mingwei Zeng, Qingyang Zhang, Jie Cui *Senior Member, IEEE*, Hong Zhong, and Fengqun Wang

Abstract—In recent years, unmanned aerial vehicles (UAVs) have shown great potential in logistics delivery due to their ability to bypass traffic congestion and adapt to complex terrains. Their high efficiency, low cost, and wide coverage make them a valuable supplement to last-mile logistics. However, third-party UAV systems operating in open environments are vulnerable to eavesdropping, tampering, and other cyber-attacks, which poses risks of sensitive information leakage. Meanwhile, warehouse nodes for sanitizing private information are widely deployed in logistics systems and need to be dynamically adjusted according to demand, which poses a challenges for the management of sanitization permissions. To address these issues, we propose a dynamic sanitizable signature with multiple sanitizers, enabling each sanitizer to independently sanitize sensitive information in signed messages, thus preserving logistics data privacy. Our scheme is applicable to UAV logistics scenarios and supports the addition and revocation of sanitizers without modifying existing keys, thereby enabling flexible and efficient permissions management. Security analysis shows that the proposed scheme ensures unforgeability, privacy preservation, and other security properties. A implementation on AmovLab Prometheus 600 UAVs demonstrates lower computational and communication overhead than existing privacy-preserving schemes, confirming its efficiency and practicality in UAV logistics systems.

Index Terms—Privacy-Preserving, Sanitizable Signature, Unmanned Aerial Vehicle Logistics.

I. INTRODUCTION

In recent years, the low-altitude economy has rapidly developed as a new economic paradigm, with unmanned aerial vehicles emerging as a core enabling technology due to their flexibility and efficiency [1]. UAV-based logistics shows significant potential [2], particularly in addressing last-mile delivery in urban environments. UAV delivery reduces delivery time through optimized aerial routes [3], [4]. Moreover, UAVs can operate autonomously around the clock and reach areas that are inaccessible to traditional couriers [5], [6]. Therefore, many companies, including Amazon [7], DHL [8], Meituan, and SF Express [9] have launched pilot deployments of UAV delivery systems. Notably, Amazon introduced Amazon Prime Air [10] to enhance customer delivery services through UAV technology. Meanwhile, the Federal Aviation Administration (FAA) [11], Google and Walmart are also actively promoting the adoption of UAVs in urban logistics.

With the development of the low-altitude economy and continued investment by logistics companies in UAV delivery, we have observed the emergence of an independently operated low-altitude last-mile logistics network. To systematically characterize this evolving trend, we abstract a system architecture for future urban UAV logistics based on typical industry practices. This architecture involves four main roles: the order initiator, the logistics company, the warehouse node, and the UAV operator. When a user submits a delivery request via a relevant online platform, the order initiator transmits the order information to the logistics company [12], which is responsible for overall coordination. The logistics company selects the optimal warehouse node for dispatch, delivers the package to a designated UAV launch site near the recipient, and finally, the UAV completes the last-mile delivery task.

To prevent orders from being forged or tampered with, and to ensure that in the event of a dispute (such as the delivered package not matching the order), the UAV can prove that the package indeed originates from the order initiator and that relevant parties can verify its source, the order initiator usually generate a digital signature over the order and transmit it and the order to the logistics company [13]. Because order data typically contains sensitive information such as package details, recipient names, and contact numbers, and UAVs are often operated by third parties in environments vulnerable to eavesdropping or tampering [14], directly transmitting the complete order may result in serious privacy leakage [15]. Therefore, before sending the order and signature to the UAV for final delivery, the warehouse selected by the logistics company must remove unnecessary sensitive fields. To prevent arbitrary modifications by the warehouse, the system should enable field-level control [16], permitting only predefined fields to be altered while ensuring the message remains verifiable under the order initiator's public key [17].

On the other hand, logistics companies typically operate numerous warehouse nodes across the country. As business scales or operational strategies evolve, these nodes may be frequently added or removed. While centralized removal of sensitive information by the logistics company may seem more straightforward [18], it may lead to performance bottlenecks and creates a potential single point of failure. To better adapt to such dynamic environments, the system should allow each warehouse node to remove sensitive information in the order independently within its own authorization scope. Additionally, it should support a highly flexible permission management mechanism, including the dynamic assignment

M. Zeng, Q. Zhang, J. Cui, H. Zhong and F. Wang are with the Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, School of Computer Science and Technology, Anhui University, Hefei 230039, China, and the Anhui Engineering Research Center for IoT Security Technologies, Anhui University, Hefei 230039, China (e-mail: qyzhang@ahu.edu.cn).

of permissions to new warehouse nodes and timely revocation of permissions for retired ones [19], thereby ensuring that sensitive information is securely and accurately managed throughout the entire unmanned aerial vehicles logistics process [20].

Sanitizable signature [21] allows an authorized sanitizer to modify designated fields in a signed message, enabling selective removal of sensitive information. However, existing schemes support only a single sanitizer, while extending them to multiple sanitizers either requires complex group signatures or separate signatures for each sanitizer, resulting in high overhead [22]. To overcome these limitations, we introduce a new sanitizable signature supporting multiple sanitizers with dynamic permission management. Our design leverages the Chinese remainder theorem to enable efficient delegation and revocation of sanitization rights [23], ensures signer traceability for accountability, and offers a scalable and practical solution for order privacy protection in logistics networks. Our main contributions are summarized as follows:

- **System Architecture.** Motivated by the development of the low-altitude economy and inspired by practical logistics operations, we propose a UAV-assisted urban logistics architecture involving order initiator, logistics companies, warehouse nodes, and third-party UAV operators. It accurately captures real-world stakeholder interactions and lays a structured foundation for the design and integration of secure and efficient functional modules.
- **Algorithm Design.** We construct a Dynamic Sanitizable Signature with Multiple Sanitizers utilizing the Chinese Remainder Theorem and Chameleon Hash. This scheme allows multiple sanitizers to independently sanitize signatures while supporting efficient permission delegation and revocation. Furthermore, it employs Non-Interactive Zero-Knowledge proofs to distinguish signature states, thereby establishing a robust accountability mechanism.
- **Privacy-Preserving Mechanism.** Addressing the conflict between data authenticity verification and user privacy protection in UAV logistics, we design a privacy-preserving mechanism specifically for UAV logistics to balance data authenticity and user privacy. It empowers warehouse nodes to independently remove sensitive user fields without invalidating signatures, enabling UAVs in untrusted environments to verify order authenticity while accessing only essential delivery instructions.
- **Performance Evaluation.** We formally prove that the proposed scheme satisfies a range of security properties. We also implement it on UAV hardware and compare it with existing approaches. Experimental results show that our scheme significantly reduces computational and communication overhead. The code is open-sourced at <https://github.com/rookie-papers/DSS-MS.git>.

Organization. The remainder of this paper is organized as follows. Section II reviews related work. Section III covers cryptographic preliminaries. Section IV presents the system model, and Section V describes the design of the proposed DSS-MS scheme. Section VI shows its integration into a UAV logistics system. Section VII provides the security analysis, and Section VIII reports experimental results and comparisons.

II. RELATED WORK

Before presenting the technical details and design rationale of our proposed scheme, we begin by reviewing the most relevant literature on sanitizable signatures and privacy-preserving mechanisms for protecting sensitive information.

Sanitizing Signature. Sanitizable signatures, first introduced by Ateniese et al. [21], allow an authorized sanitizer to modify designated parts of a signed message while preserving signature validity. In 2009, Brzuska et al. [24] formally defined the security requirements for sanitizable signatures and analyzed the relationships among various security properties. They demonstrated that the original scheme [21] satisfies five core security properties under their framework. Brzuska et al. [22] further introduced the notion of unlinkability. Unlinkability ensures that an adversary cannot determine whether multiple sanitized versions originated from the same original message. To achieve this, a construction based on group signatures was proposed. However, Fleischhacker et al. [25] later pointed out that this approach relies on group signatures with specific properties, limiting its applicability. Moreover, most suitable group signature schemes incur high computational overhead. To overcome these limitations, they proposed a new construction based on signatures with perfectly re-randomizable keys.

Camenisch et al. [26] proposed a sanitizable signature scheme based on chameleon hash functions with ephemeral trapdoors. In this design, the sanitizer, who possesses a master trapdoor, must also obtain a temporary trapdoor from the signer to find collisions and perform sanitization. This approach grants the signer fine-grained control over sanitization capabilities. Zhan et al. [27] introduced a bounded sanitizable signature scheme. Their scheme allows the signer to specify not only the modifiable blocks in the message but also the maximum number of modifications and their expiration time. This provides enhanced granularity for privacy control. However, the scheme involves many bilinear pairing operations, resulting in significant computational cost. Most recently, Yang et al. [28] proposed the notion of sanitizable blind signatures (SBS) and presented a generic construction based on structure-preserving signatures and traceable ring signatures. Leveraging SBS, they designed a privacy-preserving identity authentication and key agreement protocol, along with a seamless handover protocol named PGUS-HO, targeting privacy protection in thick-MVNO environments.

Sensitive Data Privacy Protection. In addition to foundational studies, several works have explored the application of sanitizable signatures in privacy-preserving data sharing and integrity protection.

Shen et al. [29] proposed a privacy-preserving integrity verification method for cloud storage systems. Their approach supports the sanitization of sensitive data stored on cloud servers by dividing files into blocks, individually signing each block, and enabling modifications of the signatures to conceal sensitive content. Liu et al. [30] introduced an identity-based sanitizable signature scheme to facilitate privacy-preserving data sharing and integrity protection. By eliminating the need for complex certificate management, the scheme simplifies

deployment. However, both the signing and sanitization processes involve numerous bilinear pairing operations, leading to relatively high computational overhead. Zhan et al. [27] applied their bounded sanitizable signature scheme to electronic health record (EHR) sharing. The proposed framework allows fine-grained sanitization of patient data before sharing, achieving a balance between usability and privacy in healthcare scenarios. Gao et al. [31] proposed a new cryptographic primitive called Relation Authentication Label (RAL) and developed a privacy-preserving audit proof generation mechanism based on it. This design supports efficient and secure auditing without revealing users' private data, thus extending privacy-preserving techniques to audit-related applications. Li et al. [32] presented a designated-verifier aggregate signature scheme that supports both signature aggregation and sensitive data sanitization. By integrating this scheme with blockchain, they built a high-efficiency data aggregation system tailored for industrial IoT environments, effectively combining data integration, privacy protection, and accountability.

III. PRELIMINARIES

To facilitate a clear understanding of the proposed dynamic sanitizable signature with multiple sanitizers, this section introduces the foundational concepts employed in our scheme.

Elliptic Curve Cryptography. An elliptic curve E over a prime field $\mathbb{F}_p$ is typically defined by the Weierstrass equation: $E : y^2 = x^3 + ax + b$, $a, b \in \mathbb{F}_p$, where the discriminant $\Delta = 4a^3 + 27b^2 \neq 0$. The set of points on the curve, together with a point at infinity $\mathcal{O}$, forms an abelian group. The group operation is defined as follows:

- **Point Addition:** For distinct points P and Q , $R = P + Q$ is the reflection over the x -axis of the third point where the line through P and Q intersects the curve.
- **Point Doubling:** For $P = Q$, the operation $R = 2P$ is defined as the reflection across the x -axis of the point where the curve intersects the tangent line at P .
- **Scalar Multiplication:** Given a point P and an integer k , scalar multiplication $k \cdot P$ is defined as repeated addition of P with itself k times.

Chinese Remainder Theorem (CRT). The Chinese Remainder Theorem provides a way to reconstruct integers from their remainders modulo several pairwise coprime moduli. Let $n_1, n_2, \dots, n_k$ be pairwise coprime positive integers, and let $M = \prod_{i=1}^k n_i$. Then for any integers $a_1, a_2, \dots, a_k$, the system of congruences:

$$\begin{cases} x \equiv a_1 \pmod{n_1} \\ \vdots \\ x \equiv a_k \pmod{n_k} \end{cases}$$

has a unique solution modulo M . The solution can be computed as: $x \equiv \sum_{i=1}^k a_i \cdot M_i \cdot y_i \pmod{M}$, where $M_i = \frac{M}{n_i}$ and y_i is the multiplicative inverse of M_i modulo n_i .

IV. SYSTEM ARCHITECTURE

Our system model follows the structure of the previously introduced architecture, with the following refinements. A

trusted authority is introduced to support privacy protection, and the warehouse node is treated as part of the logistics company. The system architecture is illustrated in Fig. 1.

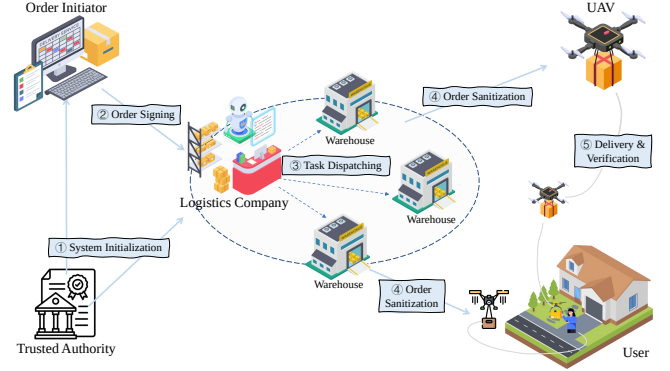


Fig. 1. A privacy-protected unmanned aerial vehicle logistics system model

- **TA (Trusted Authority):** The TA is a fully trusted entity, typically operated by government agencies or other authoritative institutions. It is responsible for generating system parameters, issuing signing keys to other entities, and providing the necessary certificates.
- **OI (Order Initiator):** The order initiator, typically an entity such as an e-commerce platform or a medical institution, receives a user's purchase request and generates an order containing details about the requested package and the user's sensitive information. It signs the order using its private key to ensure authenticity and integrity, and then forwards it to the logistics company.
- **LC (Logistics Company):** Based on the order from the order initiator, the logistics company selects a geographically optimal warehouse for dispatch. To protect user privacy, the selected warehouse sanitizes sensitive order fields (e.g., name, phone number, address) before handing the package over to the UAV. The logistics company can dynamically add or remove warehouse nodes and manage their sanitization permissions as needed.
- **UAV (Unmanned Aerial Vehicle):** UAVs are usually operated by third-party providers and are responsible for the final stage of package delivery. Since they operate in open network environments and may face potential security threats, the order data they receive contains no sensitive user information. Instead, the UAV verifies the order's integrity and authenticity before completing the delivery, ensuring that the data has not been tampered with during transmission.

As illustrated in Fig. 1, the workflow proceeds as follows: First, the Trusted Authority establishes public parameters and distributes identity certificates to all participants (Step ①). Next, the Order Initiator digitally signs the original transaction order to ensure data authenticity and verifiability (Step ②). Upon receiving the order, the Logistics Company routes it to the geographically optimal warehouse node (Step ③). To prevent privacy leakage, the warehouse masks sensitive fields (e.g., phone numbers) and updates the signature before

handing the task over to the UAV (Step ④). Finally, the UAV verifies the integrity of the sanitized data prior to physical delivery to ensure the information has not been tampered with during transmission (Step ⑤).

V. PROPOSED DYNAMIC SANITIZABLE SIGNATURE WITH MULTIPLE SANITIZERS

In this section, we present the construction of our dynamic sanitizable signature with multiple sanitizers (DSS-MS). We formalize the syntax and algorithms of DSS-MS, and define the threat model along with the desired security goals.

A. Dynamic Sanitizable Signature with Multiple Sanitizers

We propose a novel dynamic sanitizable signature with multiple sanitizers, formally defined as a set of probabilistic polynomial-time (PPT) algorithms $DSS-MS = \{Setup, KGen, Sign, Sanitizing, Verify, Proof, Judge, Join, Revoke\}$. The definitions of these algorithms are as follows:

- $pp \leftarrow Setup(1^\lambda)$: This algorithm takes as input the security parameter λ and outputs the global public parameters pp for the system.
- $(\{sk_i, PK_i\}, PK_s, S_x, sk, PK) \leftarrow KGen(pp, n)$: Given the public parameters and the number of sanitizers n , this algorithm generates a public-private key pair (sk_i, PK_i) for each sanitizer $i \in [n]$, a global sanitization public key PK_s , a sanitization key mask S_x , and a signing key pair (sk, PK) for the original signer.
- $\sigma \leftarrow Sign(pp, sk, M, PK_s)$: The signing algorithm takes as input the public parameters pp , the signing secret key sk , a message M consisting of a non-sanitizable part m_0 and a sanitizable part m , and the sanitization public key PK_s . It outputs a signature σ on the message M .
- $(M', \sigma') \leftarrow Sanitizing(\sigma, sk_i)$: A sanitizer with secret key sk_i replaces the sanitizable component m in σ with a new message m' , and computes a new valid signature σ' on the modified message $M' = (m_0, m')$.
- $b \leftarrow Verify(\sigma, PK)$: The algorithm takes as input a signature σ (which may be original or sanitized), and the signer's public key PK , outputs $b = 1$ if the signature is valid, otherwise outputs $b = 0$.
- $\pi \leftarrow Proof(\sigma, t)$: In case of a dispute, the original signer can use t to generate a proof π to demonstrate that the signature σ has been sanitized by an authorized sanitizer.
- $(sig, san) \leftarrow Judge(\pi)$: If the proof is successfully verified, indicating that the signature was generated by a sanitizer, it outputs san ; otherwise, it outputs sig .
- $(\{sk_i, PK_i\}_{i=1}^n, S_x) \leftarrow Join(n)$: This algorithm initializes n new sanitizers by generating n independent key pairs $\{(sk_i, PK_i)\}_{i=1}^n$, where $sk_i \in \mathbb{Z}_p^*$ is a private key and $PK_i = sk_i \cdot P$ is the corresponding public key. It then updates the global sanitization scalar S_x .
- $S_x \leftarrow Revoke(pp, I)$: This algorithm takes as input the public parameters and a set I of indices of the sanitizers to be revoked. It outputs a new sanitizer key mask S_x to revoke the sanitization permissions of the corresponding sanitizers without affecting other sanitizers' keys.

B. Threat Model

To analyze potential security threats, we categorize adversaries into two types: external adversary $\mathcal{A}_1$ and internal adversary $\mathcal{A}_2$, each with specific capabilities defined as follows:

$\mathcal{A}_1$: It is assumed that all communications between entities and the network occur over public channels, which $\mathcal{A}_1$ can fully monitor and manipulate. This conforms to the Dolev-Yao threat model [33], where the adversary is capable of intercepting, modifying, and fabricating messages, as well as impersonating legitimate entities. However, $\mathcal{A}_1$ is computationally bounded and cannot predict the random values generated during protocol execution, solve cryptographic hard problems, or access any secret data stored by trusted entities.

$\mathcal{A}_2$: Compared to the external adversary $\mathcal{A}_1$, $\mathcal{A}_2$ possesses stronger capabilities. In addition to inheriting all the attack methods of $\mathcal{A}_1$, it is also granted access to the private keys of one or more sanitizers. As a result, $\mathcal{A}_2$ models more complex and realistic attack scenarios.

C. Security goals

In existing research, scholars have extensively discussed the security requirements that sanitizable signatures should satisfy. Considering the application scenario of UAV logistics systems, we argue that the privacy-preserving algorithm adopted by such systems should satisfy the following security properties.

Unforgeability: Unforgeability ensures that no adversary without the signing private key can produce a valid signature on a message M . This guarantees that the packages transported by UAVs are indeed issued by legitimate sources.

Immutability: Immutability requires that authorized sanitizers can only modify the designated sanitizable parts of a message and cannot tamper with the immutable parts, thereby preventing malicious alterations to critical fields in the order.

Privacy: Privacy ensures that an adversary cannot infer any information about the original message M from the sanitized message M' and its corresponding signature σ' . This property effectively safeguards sensitive user data from being accessed by UAVs or other potential adversaries.

Transparency: Transparency requires that the original signature and its sanitized version be computationally indistinguishable in form, such that an adversary cannot determine whether a signature was generated by the original signer or produced through sanitization.

Accountability: Accountability means that when the origin of a signature is in dispute, the system should provide a mechanism to determine whether the signature was generated by the original signer or was produced through sanitization, thereby supporting responsibility attribution and security auditing.

D. Security definition

To facilitate the formal definition of our security experiments, using $Out \leftarrow \mathcal{A}^{\mathcal{O}_{Sign}(\cdot), \mathcal{O}_{Sanit}(\cdot), \mathcal{O}_{Proof}(\cdot), \dots}(\mathcal{I})$ to represent that the adversary $\mathcal{A}$, given input $\mathcal{I}$ and access to a set of oracles (such as signing oracle $\mathcal{O}_{sign}$, etc.), outputs a result Out . Using this notation, we formally define the following five security properties.

Definition 1 (Unforgeability): *If, for any probabilistic polynomial-time adversary $\mathcal{A}$, the following experiment outputs 1 with only negligible probability, then the DSS-MS scheme is said to satisfy unforgeability.*

Experiment Unforgeability $_{\mathcal{A}}^{\text{DSS-MS}}(\lambda)$

- 1: $pp = (q, \mathbb{G}, P) \leftarrow \text{Setup}(1^\lambda)$
- 2: $(\{sk_i, PK_i\}, PK, S_x) \leftarrow \text{KGen}(pp, n)$
- 3: Send $\mathcal{I} = \{pp, PK, S_x\} \cup \{(sk_i, PK_i)\}_{i=1}^n$ to $\mathcal{A}$
- 4: $(M^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}(\cdot), \mathcal{O}_{\text{Sanit}}(\cdot), \mathcal{O}_{\text{Proof}}(\cdot), \mathcal{O}_H(\cdot)}(\mathcal{I})$
- 5: **If** $(m_0^*, \dots, \sigma^*) \notin L_{\text{sig}}$ and **Verify** $(M^*, \sigma^*) = 1$ **return 1, else return 0**

In the above-mentioned experiment, the challenger begins by running the *Setup* algorithm to generate the system parameters pp , and then invokes the *KGen* algorithm to generate key pairs (sk_i, PK_i) for all sanitizers. The system public key PK and the sanitization key mask S_x are also generated during this phase. To model a strong adversary, the challenger provides the adversary $\mathcal{A}$ with the public parameters (pp, PK, S_x) , along with all sanitizer key pairs. The adversary is then allowed to adaptively query a set of oracles, including the signing oracle $\mathcal{O}_{\text{Sign}}$, the sanitization oracle $\mathcal{O}_{\text{Sanit}}$, the proof oracle $\mathcal{O}_{\text{Proof}}$, and the random oracle $\mathcal{O}_H$, in order to gather information and attempt to forge a valid signature. Eventually, the adversary outputs a message $M^* = (m_0, m)$ and a signature σ^* . If the tuple $(m_0, \dots, \sigma^*)$ does not appear in the signing query list L_{sig} and $\text{Verify}(M^*, \sigma^*) = 1$, then the adversary is considered successful in breaking the immutability property, and the experiment outputs 1; otherwise, it outputs 0.

The definitions of the remaining security experiments follow a similar structure and semantics as the one above, therefore, we omit their detailed descriptions in the following definitions.

Definition 2 (Immutability): *If, for any probabilistic polynomial-time adversary $\mathcal{A}$, the following experiment outputs 1 with only negligible probability, then the DSS-MS scheme is said to satisfy Immutability.*

Experiment Immutability $_{\mathcal{A}}^{\text{DSS-MS}}(\lambda)$

- 1: $pp = (q, \mathbb{G}, P) \leftarrow \text{Setup}(1^\lambda)$
- 2: $(\{sk_i, PK_i\}, PK, S_x) \leftarrow \text{KGen}(pp, n)$
- 3: Send $\mathcal{I} = \{pp, PK, S_x\} \cup \{(sk_i, PK_i)\}_{i=1}^n$ to $\mathcal{A}$
- 4: Send the original message, signature (M, σ) to $\mathcal{A}$
- 5: $(M^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}(\cdot), \mathcal{O}_{\text{Proof}}(\cdot), \mathcal{O}_H(\cdot)}(\mathcal{I})$
- 6: **If** $(m_0^*, \dots, \sigma^*) \notin L_{\text{sig}}$ and **Verify** $(M^*, \sigma^*) = 1$ **return 1, else return 0**

Definition 3 (Transparency): *If, for any probabilistic polynomial-time adversary $\mathcal{A}$, the following experiment outputs 1 with a probability only negligibly higher than $1/2$, then the DSS-MS scheme is said to satisfy Transparency.*

Experiment Transparency $_{\mathcal{A}}^{\text{DSS-MS}}(\lambda)$

- 1: $pp = (q, \mathbb{G}, P) \leftarrow \text{Setup}(1^\lambda)$
- 2: $(\{sk_i, PK_i\}, PK, S_x) \leftarrow \text{KGen}(pp, n)$
- 3: Send $\mathcal{I} = \{pp, PK, S_x\} \cup \{(sk_i, PK_i)\}_{i=1}^n$ to $\mathcal{A}$
- 4: Choose $b \in \{0, 1\}$, let (m_b, σ_b) be the original signed message and (m_{1-b}, σ_{1-b}) be the sanitized one.
- 5: Send both (m_b, σ_b) and (m_{1-b}, σ_{1-b}) to $\mathcal{A}$
- 6: $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}(\cdot), \mathcal{O}_{\text{Sanit}}(\cdot), \mathcal{O}_{\text{Proof}}(\cdot), \mathcal{O}_H(\cdot)}(\mathcal{I})$
- 7: **If** $b' = b$ **return 1, else return 0**

Definition 4 (Privacy): *If, for any probabilistic polynomial-time adversary $\mathcal{A}$, the following experiment outputs 1 with a probability only negligibly higher than $1/2$, then the DSS-MS scheme is said to satisfy Privacy.*

Experiment Privacy $_{\mathcal{A}}^{\text{DSS-MS}}(\lambda)$

- 1: $pp = (q, \mathbb{G}, P) \leftarrow \text{Setup}(1^\lambda)$
- 2: $(\{sk_i, PK_i\}, PK, S_x) \leftarrow \text{KGen}(pp, n)$
- 3: Send $\mathcal{I} = \{pp, PK, S_x\} \cup \{(sk_i, PK_i)\}_{i=1}^n$ to $\mathcal{A}$
- 4: Choose $b \in \{0, 1\}$, let $M(m_0, b), \sigma$ be the original signed message, (M', σ') be the sanitized one
- 5: Send both (M, σ) and (M', σ') to $\mathcal{A}$
- 6: $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}(\cdot), \mathcal{O}_{\text{Sanit}}(\cdot), \mathcal{O}_{\text{Proof}}(\cdot), \mathcal{O}_H(\cdot)}(\mathcal{I})$
- 7: **If** $b' = b$ **return 1, else return 0**

Definition 5 (Accountability): *If, for any probabilistic polynomial-time adversary $\mathcal{A}$, the following experiment outputs 1 with only negligible probability, then the DSS-MS scheme is said to satisfy Accountability.*

Experiment Accountability $_{\mathcal{A}}^{\text{DSS-MS}}(\lambda)$

- 1: $pp = (q, \mathbb{G}, P) \leftarrow \text{Setup}(1^\lambda)$
- 2: $(\{sk_i, PK_i\}, PK, S_x) \leftarrow \text{KGen}(pp, n)$
- 3: Send $\mathcal{I} = \{pp, PK, S_x\} \cup \{(sk_i, PK_i)\}_{i=1}^n$ to $\mathcal{A}$
- 4: Send the original message, signature (M, σ) to $\mathcal{A}$
- 5: $\pi \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}(\cdot), \mathcal{O}_{\text{Sanit}}(\cdot), \mathcal{O}_{\text{Proof}}(\cdot), \mathcal{O}_H(\cdot)}(\mathcal{I})$
- 6: **If** $\text{Judge}(\pi) = \text{sig}$ and (R^*, z^*) in π has not been queried to $\mathcal{O}_{\text{proof}}(\cdot)$ **return 1, else return 0**

E. Construction Based on CRT

The DSS-MS scheme supports different sanitizers in independently sanitizing the original message and its signature using their own private keys, and allows for efficient addition and revocation of sanitizers. We begin by describing each algorithm in the DSS-MS scheme in detail, followed by a schematic overview of the entire construction in Fig. 2.

Dynamic Sanitizable Signature with Multiple Sanitizers

Setup(1^λ) :

- 1: **return** $pp = (q, G, P) \leftarrow \text{ECC.Setup}()$.

KGen(pp, n) :

- 1: The trusted authority generates $\{sk_i\}_{i=1}^n, sk_s \in_R \mathbb{Z}_q^*$
- 2: Compute $N = \prod_{i=1}^n sk_i$ and $N_i = N/sk_i$. ($i \in [n]$)
- 3: Let $y_i \cdot N_i \equiv 1 \pmod{sk_i}$, $S_x = (\sum_{i=1}^n y_i \cdot N_i) \cdot sk_s$
- 4: Update parameters $pp = pp \cup S_x \cup (PK_s = sk_s P)$
- 5: Generate $sk \in_R \mathbb{Z}_q^*$, **return** (sk, PK) to signer.
- 6: **return** $(sk_i, PK_i = sk_i \cdot P)$ to sanitizer S_i

Sign($pp, M = (m_0, m)$, sk, PK_s) :

- 1: Select $r, s, t \in_R \mathbb{Z}_q^*$ and compute $R = r \cdot P, T = t \cdot P$.
- 2: Compute $e = H(m, T)$ and $CH = T + s \cdot P + e \cdot PK_s$.
- 3: Compute $c = H(m_0 || R || CH)$ and $z = r + sk \cdot c$.
- 4: **return** signature $\sigma = (R, z, s, T)$ on the message M .

Sanitizing(σ, S_x, sk_i) :

- 1: Compute $sk_s = S_x \bmod sk_i$ and select $m', k \in_R \mathbb{Z}_q^*$,
- 2: Compute $T' = CH - k \cdot P$, $s' = k - H(m', T') \cdot sk_s$.
- 3: **return** $M' = (m_0, m')$ and $\sigma' = (R, z, s', T')$.

Verify(σ', PK) :

- 1: Compute $e' = H(m', T')$, $CH = T' + s' \cdot P + e' \cdot PK_s$
- 2: **return** 1, if $z \cdot P = R + H(m_0, R, CH) \cdot PK$ hold.

Proof(M, σ, t) :

- 1: Assume the original signature $\sigma^* = (T^* = t^* \cdot P, \dots)$
- 2: **return** $\perp$ if **Verify**(M, σ) $\neq 1$, else generate $r' \in \mathbb{Z}_q^*$, and compute $R' = r' \cdot P$, and $z' = r' + H(R', T^*) \cdot t^*$.
- 3: **return** identity proof $\pi = \{M^*, \sigma^*, (R', z'), M, \sigma\}$.

Judge(π) :

- 1: **return** *san*, If $z'P = R' + H(R', T^*) \cdot T^*$ hold, and **Verify**(M^*, σ^*) = 1, and the elements in the two signatures satisfy ($R^* = R, z^* = z$); else **return** *sig*.

Join(pp, n):

- 1: TA generates $\{sk_i\}_{i=1}^{m+n}$, all sk_i pairwise coprime.
- 2: And compute $N = \prod_{i=1}^{m+n} sk_i$, $N_i = N/sk_i$.
- 3: Let y_i satisfy $y_i \cdot N_i \equiv 1 \pmod{sk_i}$ and update $S_x = (\sum_{i=1}^{m+n} y_i \cdot N_i) \cdot sk_s$. Then update $pp = pp \cup \{S_x\}$.
- 4: **return** $(sk_i, PK_i = sk_i \cdot P)$ for new sanitizer S_i .

Revoke(pp, I):

- 1: Let $I = \{i_1, \dots, i_n\}$ be revoked sanitizer indices.
- 2: $S_x = (S_x - (\sum_{i \in I} y_i \cdot N_i) \cdot sk_s) \cdot r$ and $PK_s = r \cdot PK_s$.

1) **Setup**: The TA runs $\text{ECC.Setup}(\lambda)$ to generate an elliptic curve cyclic group G of order q and selects a generator P of G . It then outputs the public parameters $pp = (q, G, P)$.

2) **Key Generation**: Given the public parameters and the number of sanitizers n , the algorithm proceeds as follows:

Sanitizer Key Generation: The TA randomly generates pairwise coprime private key $sk_i \in_R \mathbb{Z}_q^*$ for each sanitizer S_i (where $i = 1, \dots, n$) and computes the public key $PK_i = sk_i \cdot P$. After generating all private keys, it computes the product $N = \prod_{i=1}^n sk_i$. For each sanitizer S_i , the TA derives $N_i = N/sk_i$, and determines y_i such that $y_i \cdot N_i \equiv 1 \pmod{sk_i}$.

Signer Key Generation: The TA generates signer's secret key $sk \in_R \mathbb{Z}_q^*$ and computes the public key $PK = sk \cdot P$.

Temporary Sanitization Key Generation: The TA first generates a temporary sanitization secret key $sk_s \in_R \mathbb{Z}_q^*$ and derives the public key $PK_s = sk_s \cdot P$. It then computes $S_x = (\sum_{i=1}^n y_i \cdot N_i) \cdot sk_s$. Finally, the TA updates the public parameters by adding PK_s and S_x , and distributes the key pairs to the appropriate entities via a secure channel.

3) **Computing the Original Signature**: To sign a message $M = (m_0, m)$, where m_0 is the immutable part and m is the sanitizable part. The signer first generates three random values $r, s, t \in_R \mathbb{Z}_q^*$ and computes $R = r \cdot P, T = t \cdot P$. Next, it computes a hash value $e = H(m, T)$ and the commitment $CH = T + s \cdot P + e \cdot PK_s$. Another hash $c = H(m_0 || R || CH)$ is then calculated, and the response is set as $z = r + sk \cdot c$. The final signature on message M is $\sigma = (R, z, s, T)$. The signer retains t to prove that they are the original signer.

4) **Sanitizing the Original Signature**: To sanitize the original signature $\sigma = (R, z, s, T)$ associated with the message $M = (m_0, m)$, a sanitizer holding secret key sk_i derives the sanitization secret key as $sk_s = S_x \bmod sk_i$. Suppose the sanitizer wishes to update the sanitizable message m to m' . It randomly generates a scalar $k \in_R \mathbb{Z}_q^*$ and computes $T' = CH - k \cdot P$. Then it computes $e' = H(m', T')$ and $s' = k - e' \cdot sk_s$. The sanitized message becomes $M' = (m_0, m')$, and the resulting sanitized signature is $\sigma' = (R, z, s', T')$.

5) **Signature Verification**: To verify a (possibly sanitized) signature $\sigma = (R, z, s, T)$ with respect to the public key PK , the verifier begins by computing the hash $e = H(m, T)$ and forming the commitment $CH = T + s \cdot P + e \cdot PK_s$. Then it computes $c = H(m_0, R, CH)$ and checks whether $z \cdot P \stackrel{?}{=} R + c \cdot PK$ holds. If the equality holds, the verifier accepts the signature and returns 1; otherwise, it returns 0.

6) **Generating the Proof**: In practice, a sanitizer might negligently fail to sanitize a sensitive message, causing a privacy breach. To evade responsibility, the sanitizer might claim that the leaked message and signature were generated by the signer. Conversely, a signer who has leaked the original message and signature may claim that the leaked information is a sanitized signature and accuse the sanitizer. To prevent such disputes, we introduce an accountability mechanism.

The core idea is that the signer knows the discrete logarithm t^* corresponding to T^* in the original signature $\sigma^* = (R, z, s^*, T^*)$, while the sanitizer, after replacing T^* with T during sanitization, does not know the corresponding t . Therefore, the signer can prove σ^* is original signature via a zero-knowledge proof $\text{NIZK} = \{t^* : T^* = t^* P\}$.

Fig. 2. Dynamic Sanitizable Signature with Multiple Sanitizers (DSS-MS)

Furthermore, since only the sanitizer can modify a message-signature pair while retaining verifiability, the signer can establish accountability by providing the following:

- The original message M^* and its signature σ^* ;
- The sanitized message M and its signature σ ;
- Zero-knowledge proof $R', z' \leftarrow \text{NIZK} : \{t^* : T = t^*P\}$.

If $\text{Verify}(M^*, \sigma^*) = \text{Verify}(R', z') = 1$ and the elements in two signatures σ and σ^* satisfy $(R^* = R, z^* = z)$, we conclude that M^* is the original message and σ^* is the signer-generated original signature, while M and σ are the sanitized message and signature, respectively. Therefore, the sanitized signature σ must have been generated by the sanitizer. If the signer fails to provide such a proof, the signature is considered self-generated, and the signer bears the responsibility.

The proof π is generated as follows: the signer selects a random scalar $r' \in_R \mathbb{Z}_q^*$ and computes the commitment $R' = r' \cdot P_1$. Then, the signer computes the challenge $c' = H(R', T^*)$ and the response $z' = r' + c' \cdot t^*$. The resulting proof is (R', z') . Return the proof $\pi = \{M^*, \sigma^*, (R', z'), M, \sigma\}$.

Remark: It is worth noting that the signer cannot claim that a leaked original signature $\sigma^* = (R, z, s^*, T^*)$ was generated by the sanitizer. To do so, the signer would need to construct a new message m' and a forged signature $\sigma = (R, z, s, T)$ such that $(s, T) \neq (s^*, T^*)$ and the chameleon hash function satisfies the collision condition: $\text{CH}(m^*, s^*, T^*) = \text{CH}(m, s, T)$, so that the forged signature still verifies. This is equivalent to finding a collision in the chameleon hash function without knowing the sanitizer's private key sk_s . As we prove in Section VII, our chameleon hash is collision-resistant, making such attacks computationally infeasible.

7) **Judging the Origin of the Signature:** To verify the origin of a signature σ , a verifier (e.g., an auditor or judge) checks if $\text{Verify}(M^*, \sigma^*) = \text{Verify}(R', z') = 1$ and whether σ and σ^* satisfy $(R^* = R, z^* = z)$. If so, the signature is deemed from the sanitizer, return `san`; otherwise, it is confirmed to be generated by the signer, return `sig`.

8) **Join New Sanitizers:** Suppose there are n new users who wish to obtain sanitization permission. The trusted authority randomly generates new private keys $\{sk_{m+i}\}_{i=1}^n$, ensuring that $\gcd(sk_i, sk_j) = 1$ for all $(i \neq j) \in [m+n]$, where m is the number of existing sanitizers. Then, the trusted authority computes $N = \prod_{i=1}^{m+n} sk_i$, and for each sanitizer S_i , it calculates $N_i = N/sk_i$ and finds $y_i = N_i^{-1} \bmod sk_i$. The updated S_x is computed $S_x = (\sum_{i=1}^{m+n} y_i \cdot N_i) \cdot sk_s$. Finally, the trusted authority securely distributes each newly generated secret key sk_i to the corresponding sanitizer.

9) **Revoke Sanitizers:** When certain sanitizers are no longer authorized or need to be revoked, let $I = \{idx_1, \dots, idx_n\}$ denote the set of indices corresponding to the sanitizers to be revoked. The TA updates $S_x = (S_x - (\sum_{i \in I} y_i \cdot N_i) \cdot sk_s) \cdot r$. As a result, when the signer subsequently uses the updated sanitization public key $PK'_s = r \cdot PK_s$ to sign a message, the revoked sanitizers can no longer recover $sk'_s = r \cdot sk_s$ to sanitize the signature using their secret keys sk_i .

VI. PRIVACY-PROTECTED UAV LOGISTICS SYSTEM

To achieve privacy protection for user order information during UAV logistics transmission, this section presents a

complete operational workflow of a privacy-preserving UAV logistics system. The system involves four main entities: the trust authority, the order initiator, logistics company with warehouse nodes, and third-party UAVs operators. The overall process consists of several phases, including system setup, entity registration, signature generation, privacy sanitization, signature verification, traceability, and sanitizer management.

1) **System Initialization:** The trusted authority runs the *DSS-MS.Setup* algorithm to generate the system's public parameters (q, G, P) , which are then published to provide the foundational setup for all subsequent operations in the system.

2) **Entity Registration:** To join the privacy-protected UAV logistics system, the order initiator, logistics company, and third-party UAVs must register with the trusted authority to obtain their key pairs and corresponding certificates. The registration process for each entity is as follows:

- **Order Initiator Registration:** The order initiator randomly generates a private key $sk \in \mathbb{Z}_q^*$ and computes public key $PK = sk \cdot P$. It then submits PK to the TA, which issues a certificate to validate its identity.
- **Logistics Company Registration:** Suppose the logistics company operates n geographically distributed warehouse nodes, denoted as $W_1, \dots, W_n$. The company generates pairwise coprime private key $sk_i \in_R \mathbb{Z}_q^*$ for each warehouse W_i and computes the corresponding public key as $PK_i = sk_i \cdot P$. The logistics company first computes $N = \prod_{i=1}^n sk_i$. Then, for each warehouse W_i , it calculates $N_i = N/sk_i$ and determines the modular inverse y_i such that $y_i \cdot N_i \equiv 1 \pmod{sk_i}$. Finally, the LC computes $S_x = (\sum_{i=1}^n y_i \cdot N_i) \cdot sk_s$ and publishes it as part of the system's public parameters to allow any warehouse to perform signature sanitization.
- **UAV Registration:** Any third-party UAV can register with the TA to become a legitimate logistics participant. Each UAV selects a private key $usk \in \mathbb{Z}_q^*$ and computes the corresponding public key $UPK = usk \cdot P$. The public key is submitted to the TA to obtain a certificate.

3) **Signature Generation:** After a user places an order on the order initiator (e.g. e-commerce platform), the order initiator signs the order to ensure its authenticity and integrity. The order typically contains private user information such as name, phone number, address, and product details. To support privacy protection, the order is divided into an immutable part m_0 and a sanitizable part m (e.g., name and phone number).

The order initiator (OI) uses its private key sk to execute *DSS-MS.Sign*($pp, M = (m_0, m), sk, PK_s$) to generate the signature $\sigma = (R, z, s, T)$ and retains the randomness t used during signing for future accountability. Finally, the OI sends the order M and the signature σ to the logistics company.

4) **Sanitizing Sensitive Information:** Upon receiving an order, the logistics company selects a warehouse W_i based on the package details, typically choosing the one geographically closest to the recipient's address. The order and its signature are then forwarded to W_i , which delegates the final delivery to a UAV. To reduce the risk of privacy leakage in case of UAV compromise, the W_i sanitizes the order by removing sensitive information while retaining only the fields required for delivery. Specifically, it replaces the sensitive information m with

m' (where sensitive fields are masked with ***), executes the *DSS-MS.Sanitizing* algorithm to generate a sanitized signature σ' , and forwards the tuple (M', σ') to the UAV.

5) **Signature Verification:** The UAV verifies the received order's authenticity before proceeding with delivery. Specifically, upon receiving $\{M', \sigma' = (R, z, s', T')\}$, the UAV computes $e = H(m', T')$ and reconstructs the chameleon hash $CH = T' + s' \cdot P + e \cdot PK_s$. It then computes the challenge $c = H(m_0, R, CH)$ and checks whether the verification equation $z \cdot P = R + c \cdot PK$ holds. If the equation holds, the signature is accepted and the UAV proceeds with the delivery.

6) **Traceability:** In the event of a privacy breach involving order information, where both the order initiator and the logistics company deny responsibility, it becomes essential to determine the origin of a given order M and signature σ . Since the order initiator knows the original order message and signature $(M^*, \sigma^* = (R, z, s^*, T^*))$, as well as the discrete logarithm t^* such that $T^* = t^* \cdot P$, it can generate a proof of accountability $\pi = \{M^*, \sigma^*, (R', z'), M, \sigma\}$ using *DSS-MS.Proof* (M, σ, t^*) , where (R', z') constitutes a zero-knowledge proof $\text{NIZK} : \{t^* : T^* = t^* \cdot P\}$.

To verify whether the signature originates from the OI, the verifier checks whether $\text{Verify}(M^*, \sigma^*) = 1$, and the equation $z'P = R + H(m', T^*) \cdot T^*$ hold, and the corresponding elements of σ and σ^* satisfy $(R^* = R, z^* = z)$. If these conditions are met, output *san*; otherwise, output *sig*.

7) **Sanitizer Authority Management:** When a logistics company needs to add n new warehouses to expand service capacity, it must grant sanitization authority to these entities. This is achieved by executing the *DSS-MS.Join* algorithm, which generates unique sanitization private keys $\{sk_{m+i}\}_{i=1}^n$ for the new warehouses. Each newly generated key sk_i is then securely distributed to the corresponding sanitizer, enabling them to participate in future sanitization operations.

Conversely, when certain warehouses are no longer required or their privileges must be revoked, the logistics company identifies the set of indices $I = \{idx_1, \dots, idx_n\}$ corresponding to the target sanitizers and runs the *DSS-MS.Revoke* algorithm. This process removes the revoked entities' contributions from the global sanitization key mask S_x and updates the sanitization key to $PK'_s = r \cdot PK_s$, thereby ensuring that revoked sanitizers can no longer recover the updated sanitization secret $sk'_s = r \cdot sk_s$.

VII. SECURITY PROOF AND ANALYSES

We first prove that our scheme satisfies correctness. Then, we formally show that it meets the required security properties. We begin by introducing the underlying hardness assumptions, followed by the definition and analysis of each property.

A. Correctness

We claim that the proposed DSS-MS scheme satisfies correctness. Suppose the signer generates a signature σ using the *Sign* algorithm, and the sanitizer produces a sanitized signature σ' using the *Sanit* algorithm. Then the verification algorithm always accepts both signatures, i.e., $\text{Verify}(\sigma', PK) = 1$ holds. The correctness is shown as follows.

The correctness of the original signature is immediate. Let $M = (m_0, m)$ be the original message and let $\sigma = (R, z, s, T)$ denote the signature generated by the *Sign* algorithm. During verification, the computed commitment is $CH' = T + s \cdot P + e \cdot PK_s = CH$, which leads to the same challenge value $c' = H(m_0, R, CH') = H(m_0, R, CH) = c$. Therefore, the verification equation holds as $z \cdot P = (r + c \cdot sk) \cdot P = R + c \cdot PK$.

Now consider the sanitized message $M' = (m_0, m')$ and its corresponding signature $\sigma' = (R, z, s', T')$. Compute $e' = H(m', T')$. Then the verifier evaluates:

$$\begin{aligned} CH' &= T' + s' \cdot P + e' \cdot PK_s \\ &= (CH - k \cdot P) + (k - e' \cdot sk_s) \cdot P + e' \cdot PK_s \\ &= CH \end{aligned}$$

Consequently, $c' = H(m_0, R, CH') = H(m_0, R, CH) = c$ and once again the verification equation is satisfied as $z \cdot P = (r + c \cdot sk) \cdot P = R + c \cdot PK$. Thus, both the original and sanitized signatures successfully pass verification, confirming the correctness of the DSS-MS scheme.

B. Security Assumption

To establish the security of the proposed scheme, we begin by presenting two commonly used security assumptions, which will serve as the foundation for the subsequent security proofs.

Elliptic Curve Discrete Logarithm Problem (ECDLP): Given two points P and $Q = xP$ in a cyclic group G defined over an elliptic curve, the ECDLP assumption states that it is computationally infeasible for any probabilistic polynomial-time (PPT) adversary to recover the scalar $x \in \mathbb{Z}_q$. Formally,

$$\Pr[\mathcal{A}(P, Q) = x \mid Q = xP] \leq \epsilon,$$

where ϵ is a negligible function in the security parameter.

C. General Forking Lemma [34]

Let $\mathbb{F}$ be a finite set with $|\mathbb{F}| \geq 2$, and let $q \in \mathbb{N}$ be a positive integer. Suppose x is sampled from some fixed distribution over a finite domain, and let $(h_1, \dots, h_q) \in \mathbb{F}^q$ be chosen uniformly at random. Consider a probabilistic polynomial-time algorithm $\mathcal{A}$ which, given input $(x, h_1, \dots, h_q)$, returns a pair (J, σ) , where $J \in \{0, \dots, q\}$ and σ is some associated output.

Define the success probability of $\mathcal{A}$ as $\text{acc} := \Pr[J \geq 1]$. Now define the forking experiment $F_{\mathcal{A}}(x)$ as follows:

- 1) Uniformly sample $(h_1, \dots, h_q) \in \mathbb{F}^q$.
- 2) Run $\mathcal{A}(x, h_1, \dots, h_q)$ to obtain (J, σ) .
- 3) If $J = 0$, return $(0, \perp, \perp)$.
- 4) Independently sample fresh $(h'_J, \dots, h'_q) \in \mathbb{F}^{q-J+1}$.
- 5) Run $\mathcal{A}(x, h_1, \dots, h_{J-1}, h'_J, \dots, h'_q)$ to obtain (J', σ') .
- 6) If $J = J'$ and $h_J \neq h'_J$, return $(1, \sigma, \sigma')$; otherwise, return $(0, \perp, \perp)$.

Let frk be the probability that $F_{\mathcal{A}}(x)$ returns $(1, \sigma, \sigma')$. Then the following bound holds:

$$\text{frk} \geq \text{acc} \cdot \left(\frac{\text{acc}}{q} - \frac{1}{|\mathbb{F}|} \right).$$

D. Security Proof

We adopt different adversary models to prove various security properties. Specifically, we use the internal adversary $\mathcal{A}_2$ from our threat model to prove unforgeability, immutability, and accountability, and the external adversary $\mathcal{A}_1$ to prove transparency and privacy.

Theorem 1: Suppose there exists an adversary $\mathcal{A}$ that can forge a valid signature in the proposed DSS-MS scheme with time complexity $O(T)$ and success probability ε . Then, there exists a reduction algorithm $\mathcal{B}$ that solves the ECDLP with probability at least $\varepsilon \cdot \left(\frac{\varepsilon}{q_H} - \frac{1}{2q}\right)$ within time $O(T) + O(1)$.

Unforgeability Proof. Our proposed DSS-MS satisfies unforgeability, meaning that any probabilistic polynomial-time adversary without knowledge of the signing private key sk cannot forge a signature σ on any message M such that $\text{Verify}(M, \sigma) = 1$. $\mathcal{A}_2$ in the threat model is used in this proof.

We formally define this notion of security via a standard unforgeability experiment and prove the security in the Random Oracle Model (ROM). During the experiment, the adversary can access the following oracles: a signing oracle, a sanitizing oracle, a proof oracle, and hash oracles, which together reflect all possible observable behaviors in a real-world setting. We assume the adversary makes at most $q_{\text{sign}}, q_{\text{san}}, q_{\text{prf}}$, and q_H queries to the respective oracles, where $q_H = q_{H_0} + q_{H_1}$ represents the total number of hash queries.

We now describe the interaction between the challenger and the adversary in the security game and define the winning condition for the adversary.

Initialization. The challenger runs $\text{ECCGen}(\lambda)$ to generate elliptic curve parameters (q, G, P) , and sends the public parameters $pp = (q, G, P)$ to the adversary. Then, using the key generation algorithm, the challenger generates key pairs (sk_i, PK_i) for each sanitizer, as well as a temporarily sanitized key pair $sk_s, PK_s = sk_s \cdot P$. All public keys are revealed to the adversary. It is assumed that the adversary has corrupted all sanitizers and thus knows all corresponding private keys $\{sk_i\}$.

Query phase. To simulate the adversary's ability to intercept messages over the network—such as original signatures, sanitized signatures, and other related data—we provide a set of query oracles that model these interactions.

a) **Signing Oracle Query.** Upon receiving a public key PK and a message $M = (m_0, m)$, the challenger simulates a signature on M under PK as follows:

- 1) Randomly generate $s, t \in \mathbb{Z}_q^*$ uniformly at random and compute $T = t \cdot P$.
- 2) If (m, T) has not been queried to H_0 , choose $e \in \mathbb{Z}_q$ uniformly at random, set $H_0(m, T) = e$, and record $(m, T; e)$ in list L_{H_0} ; otherwise use the existing e .
- 3) Challenge then computes $CH = T + s \cdot P + e \cdot PK_s$.
- 4) If (m_0, R, CH) is not in list L_{H_1} , choose $c, z \in \mathbb{Z}_q$, set $R = z \cdot P - c \cdot PK$, and record $(m_0, R, CH; c)$ in L_{H_1} ;
- 5) Challenge stores the tuple $(m_0, m; \sigma)$ in L_{sign} , and return the simulated signature $\sigma = (R, z, s, T)$.

b) **Sanitizing Oracle Query.** The adversary submits a forged signature $\sigma = (R, z, s, T)$ along with a modified

message m' . Given the known sk_s , the challenger simulates the sanitization process as follows:

- 1) Challenge computes the chameleon hash value $CH = T + s \cdot P + e \cdot PK_s$, where $e = H_0(m, T)$.
- 2) Randomly generate $k \in \mathbb{Z}_q^*$, compute $T' = CH - k \cdot P$.
- 3) If (m', T') has not been queried to H_0 , Randomly generate $e' \in \mathbb{Z}_q$ and store $(m', T'; e')$ to L_{H_0} . Otherwise, retrieve e' from L_{H_0} . Then compute $s' = k - e' \cdot sk_s$.
- 4) Finally, return the sanitized signature $\sigma' = (R, z, s', T')$, and record the tuple (m_0, m', σ') in L_{sig} .

c) **Proof Oracle Query.** When the adversary submits a signature σ and requests a zero-knowledge proof, the challenger proceeds as follows:

- 1) Check if σ appears in the signing record list L_{sig} . If not, reject the query (as it is a sanitized signature).
- 2) If σ is valid, retrieve the associated random value t and choose $r \in \mathbb{Z}_q$ randomly, Then compute $R = r \cdot P$.
- 3) Check whether (R, T) has been queried to H_2 . If not, set $H_2(R, T) = c$ with $c \in \mathbb{Z}_q$ chosen randomly.
- 4) Finally, compute $z = r + c \cdot t$ and return the final proof $\pi = \{M^*, \sigma^*, (R^*, z^*), M, \sigma\}$.

d) **Hash Oracle Queries.** For all hash queries to H_0, H_1, H_2 , the challenger maintains hash tables L_{H_i} . If the queried item exists, the challenger returns the recorded value; otherwise, it selects a random value and stores it.

Forgery Phase. The adversary outputs a forged signature $\sigma^* = (R^*, z^*, s^*, T^*)$ on the message $M^* = (m_0^*, m^*)$. If σ^* has not been previously recorded in the query list L_{sig} , and $\text{Verify}(M^*, \sigma^*) = 1$, then the adversary is considered successful, and the experiment outputs 1.

Security Reduction. Assume the adversary succeeds in the experiment with non-negligible probability ε . We construct a reduction algorithm $\mathcal{B}$ that uses the adversary $\mathcal{A}$ to solve the elliptic curve discrete logarithm problem (ECDLP).

By invoking the General Forking Lemma, we can run $\mathcal{A}$ twice with the same randomness and obtain two valid signatures $\sigma = (R, z, \cdot)$ and $\sigma' = (R, z', \cdot)$ such that the corresponding hash values satisfy $c \neq c'$. Since both signatures pass verification, we have: $z \cdot P = R + c \cdot PK$, $z' \cdot P = R + c' \cdot PK$. Subtracting the two equations gives: $(z - z') \cdot P = (c - c') \cdot PK$, which implies: $sk = \frac{z - z'}{c - c'} \mod q$. Thus, $\mathcal{B}$ successfully computes the private key corresponding to PK , contradicting the assumed hardness of ECDLP.

The adversary runs in time $O(T)$ and makes at most $q_{\text{sign}}, q_{\text{san}}, q_{\text{prf}}, q_H$ queries in total. Then, the total running time of the reduction $\mathcal{B}$ is $O(T + q_H + q_{\text{sign}} + q_{\text{san}} + q_{\text{prf}})$. By the General Forking Lemma, the success probability of $\mathcal{B}$ is at least: $\text{frk} \geq \varepsilon \cdot \left(\frac{\varepsilon}{q_H} - \frac{1}{2q}\right)$. Therefore, if ε is non-negligible, so is frk , and $\mathcal{B}$ breaks ECDLP in polynomial time, leading to a contradiction. Hence, our DSS-MS signature scheme achieves unforgeability in the Random Oracle Model.

Immutability Proof. The immutability property ensures that the sanitizer is restricted to modifying only the explicitly designated sanitizable fields of the signed message. Any unauthorized alterations to the immutable parts are strictly disallowed. We prove that our scheme satisfies the immutability property in the presence of an internal adversary $\mathcal{A}_2$.

Our proposed DSS-MS scheme satisfies immutability. Its security can be derived from the already proven unforgeability. Specifically, if an adversary can successfully output a message-signature pair $(M^* = (m_0^*, m^*), \sigma^*)$ such that m_0^* has not been authorized for modification (i.e., is not in the signing list $\mathcal{L}_{\text{sig}}$), and the signature verifies correctly, then the adversary has, by definition, violated immutability. In this case, the adversary forges a valid signature without querying the $\mathcal{O}_{\text{Sanit}}(\cdot)$ oracle, which directly contradicts the unforgeability of the DSS-MS. Therefore, the immutability property is upheld.

Theorem 2: *The chameleon hash embedded in our signature, defined as $CH(m, s, T) = T + sP + H(m, T) \cdot PK_s$, is collision-resistant under the assumption that the ECDLP is hard in the Algebraic Group Model (AGM) [35].*

Accountability Proof. Accountability requires that the system be able to distinguish whether a given signature was produced by the original signer or by a sanitizer. As described in Section V-E, this is achieved by allowing the signer to generate a non-interactive zero-knowledge (NIZK) proof demonstrating knowledge of the discrete logarithm of T in the original signature $\sigma = (R, z, s, T)$. Our accountability analysis proceeds in two parts: we first prove that an adversary cannot forge such a proof, and then show that the original signer cannot fabricate evidence to falsely implicate a sanitizer. The latter reduces to the collision resistance of the underlying chameleon hash.

Unforgeability of the zero-knowledge proof. Even an adversary possessing a sanitizer's private key cannot derive the discrete logarithm t of $T = tP$ from the chameleon hash $CH = T + s \cdot P + e \cdot PK_s$, as t remains unknown. Therefore, producing a valid proof essentially requires forging a Schnorr signature. Since (R, z) constitutes a Schnorr signature, which is proven to be unforgeable under the ECDLP assumption, no polynomial-time adversary can generate a valid proof π without solving the ECDLP. Thus, only the original signer who knows t can produce a valid NIZK proof, enabling them to prove whether a signature has been sanitized.

Inability to falsely implicate a sanitizer. The signer cannot fabricate a fake "original signature" after intentionally leaking the sanitized message and signature, in an attempt to falsely blame the sanitizer. If this were possible, it would contradict the collision resistance of the chameleon hash function. We prove the collision resistance of our chameleon hash in the Algebraic Group Model (AGM) [35], thereby ensuring that the signer cannot generate two different message-signature pairs that hash to the same commitment.

First, we present the formal security definition of the collision resistance of the chameleon hash function, which can be described using the model as follow:

Definition 6 (Collision Resistance): *If, for any probabilistic polynomial-time adversary $\mathcal{A}$, the following experiment outputs 1 with only negligible probability, then the Chameleon hash is said to satisfy collision resistance.*

We now describe the interaction between the challenger and the adversary in the security game and define the winning condition for the adversary.

Initialization. The challenger runs $ECCGen(\lambda)$ to generate (q, G, P) and receives the sanitizer's public key $PK = xP$,

Experiment CollisionResistance $_{\mathcal{A}}^{\text{DSS-MS}}(\lambda)$

- 1: $pp = (q, G, P) \leftarrow \text{Setup}(1^\lambda)$
- 2: $(PK_s) \leftarrow \mathbf{KGen}(pp, n)$, send $\mathcal{I} = \{pp, PK_s\}$ to $\mathcal{A}$
- 3: Send the original Chameleon hash (m, s, T) to $\mathcal{A}$
- 4: $(m', s', T', \vec{a}) \leftarrow \mathcal{A}^{\mathcal{O}_H(\cdot), \mathcal{O}_{\text{Adapt}}(\cdot)}(\mathcal{I})$
- 5: If the equation $CH(m, s, T) = CH(m', s', T')$ hold and $(m', s', T') \notin \mathcal{L}_{CH}$, **return** 1; **else return** 0.

where the challenger does not know the corresponding secret key x . Then, the challenger randomly generates a message $m \in \mathbb{Z}_q^*$ and random values $s, t \in \mathbb{Z}_q^*$, computes $T = tP$ and $CH = T + sP + H(m, T) \cdot PK$, and sends the public parameters (q, G, P) and the tuple (m, s, T) to the adversary.

Query Phase. To simulate the adversary's ability to intercept messages over the network, the challenger provides a set of query oracles modeling such interactions.

e) *Hash Oracle Queries.*: For all hash queries to H_0, H_1, H_2 , the challenger maintains corresponding hash tables L_{H_i} . If the queried item already exists, the challenger returns the recorded value; otherwise, it selects a random value uniformly from the output space and stores it in the list.

f) *Adapt Oracle Queries.*: If the adversary submits a query (m, s, T) corresponding to $CH = CH(m, s, T)$ and requests a collision message m' , the challenger answers as follows: It randomly selects $s', e' \in \mathbb{Z}_q^*$, then computes $T' = CH - s'P - e'PK$, and stores (m', T') in L_H . This response is computationally indistinguishable from the real Adapt algorithm, following the same simulation strategy as in the Schnorr signature scheme, and thus the proof is omitted here. Finally, the tuple (m', s', T') is recorded in the list L_{CH} .

Forgery Phase. The adversary $\mathcal{A}$ outputs a forged collision tuple (m^*, s^*, T^*) , and in the Algebraic Group Model (AGM) [35], it additionally outputs a vector of coefficients $\vec{a} = (a_1, \dots, a_n)$ such that

$$T^* = \sum_{i=1}^n a_i P + a_2 PK + a_3 CH_1 + \dots + a_{n+2} CH_n,$$

and $(m^*, s^*, T^*) \notin L_{CH}$. If $CH(m^*, s^*, T^*) = CH$, then the adversary is considered successful and return 1.

Security Reduction. Assume the adversary succeeds in the above experiment with non-negligible probability ε . We construct a reduction algorithm $\mathcal{B}$ that uses the adversary $\mathcal{A}$ to solve the Elliptic Curve Discrete Logarithm Problem.

Since $CH(m, s, T) = CH(m^*, s^*, T^*) = CH$, we have:

$$T + sP + ePK = T^* + s^*P + e^*PK,$$

where $e = H(m, T)$ and $e^* = H(m^*, T^*)$, both of which can be retrieved from L_H . Since T was generated by the challenger, it knows t such that $T = tP$. Suppose each $CH_i = ch_i P$, then T^* can be rewritten as:

$$T^* = a_1 P + \sum_{i=1}^n ch_i P + a_2 PK.$$

Let $A = a_1 + \sum_{i=1}^n ch_i$, then $T^* = AP + a_2 PK$. Substitute into the equality: $(t + s)P + e \cdot xP = (A + s^*)P + e^* \cdot xP$. By rearranging the terms, we can solve for x :

$$(t + s - A - s^*)P = (a_2 + e^* - e) \cdot xP.$$

$$x = \frac{t + s - A - s^*}{a_2 + e^* - e}.$$

This allows the challenger to recover the secret key x from the public key $PK = xP$, thereby solving the ECDLP — a contradiction. From the above analysis, it can be seen that the reduction algorithm $\mathcal{B}$ incurs only an additional $O(1)$ overhead and solves the ECDLP problem with the same success probability ε as the adversary's forgery. Therefore, the chameleon hash function is collision-resistant under the ROM and AGM assumptions.

Theorem 3: *The distributions of original and sanitized signatures in our scheme are computationally indistinguishable.*

Transparency Proof. Transparency ensures that the distribution of a sanitized signature is indistinguishable from that of an original signature. That is, a polynomial-time adversary cannot determine whether a given signature was created by the original signer or transformed by a sanitizer. To analyze the indistinguishability, we compare the distributions of (s, T) in the original signature and (s', T') in the sanitized signature:

Original Signature (s, T) : The signer selects $t \in_R \mathbb{Z}_p$ and computes $T = t \cdot P$, which is uniformly random in group G . The value s is also uniformly sampled from $\mathbb{Z}_p$, so (s, T) is drawn from a uniform distribution over $\mathbb{Z}_p \times G$.

Sanitized Signature (s', T') : The sanitizer chooses $k \in_R \mathbb{Z}_p$, computes $T' = CH - k \cdot P$, and then derives $s' = k - e' \cdot sk_s$ where $e' = H(m', T')$. Under the random oracle model, e' is uniform and independent, and since sk_s is hidden and k is random, s' is uniformly random to the adversary.

Hence, the sanitized signature (s', T') is indistinguishable from the original (s, T) , and transparency is satisfied.

Privacy Proof. Privacy requires that no adversary can recover any sensitive information removed by the sanitizer. As discussed in [24], this property naturally follows from transparency. If an adversary $\mathcal{A}$ is able to recover any sensitive information from a sanitized message-signature pair (M', σ') , this implies that $\mathcal{A}$ can distinguish whether the signature was generated by the original signer or produced via sanitization. Such a capability violates the transparency property, which ensures that sanitized and original signatures are computationally indistinguishable. Therefore, the satisfaction of transparency necessarily implies the satisfaction of privacy.

VIII. EXPERIMENTAL EVALUATION

To assess the performance of our proposed approach, we conducted a comprehensive evaluation and compared it against three representative schemes: BFLS10 by Brzuska et al. [22], DVAS by Li et al. [32], and IBSDIA by Liu et al. [36]. The evaluation focuses on both computation and communication overhead. All source code will be made publicly accessible at: <https://github.com/rookie-papers/DSS-MS.git>

A. Experimental Environment

We implemented both our proposed and comparison schemes in C++. The order initiator and warehouse nodes were deployed on a PC, while UAV experiments were conducted on AmovLab Prometheus 600 UAVs [37], with communication enabled through Homer modules, as illustrated in Fig. 3.



Fig. 3. Prometheus 600 UAV platform used in our evaluation

The PC was equipped with an 11th Gen Intel(R) Core(TM) i7-11700K @ 3.60GHz processor, 8 GB RAM, dual CPUs, and an 80 GB hard disk, running Ubuntu 24.04.1. The P600 UAV platform featured an 8-core ARM64 Cortex-A78AE processor and 8 GB RAM. Cryptographic functionalities in our implementation were supported by the Miracl Core library [38], utilizing the BLS12381 elliptic curve, which ensures a security strength close to 128 bits. To address the precision and range limitations of the BIG class in Miracl Core, we employed the GMP library for large number computations, converting between GMP and BIG as needed. SHA256 was adopted as the hash function. We evaluated the computational overhead using the Google Benchmark framework [39]. Communication among system components was realized via WebSocket.

B. Computation Cost Analysis

We begin by introducing the notations used in our computational cost analysis. Specifically, scalar multiplication and point addition in elliptic curve groups G_1 are denoted by G_1^m and G_1^a , respectively. The cost of a standard hash function is represented by H , while H_p denotes the time cost of hashing to a point on the elliptic curve. We use T_p to represent the cost of a bilinear pairing operation, and G_T^p denotes exponentiation in the target group G_T . These notations are used throughout the performance evaluation for clarity and consistency.

To comprehensively evaluate the performance of the proposed scheme, we compare our design with three representative schemes from both theoretical and experimental perspectives. The comparison covers four essential phases: signing, sanitization, verification, and tracing, where tracing refers to the total time required for both generating and verifying the proof. In our theoretical analysis, we estimate the main computational costs of each scheme. To maintain conciseness, we focus only on expensive operations such as scalar multiplications, pairings, and hash-to-curve computations, while

TABLE I
TIME-CONSUMING CRYPTOGRAPHIC OPERATIONS

Scheme Phase	BFLS10 [22]	DVAS [32]	IBSDIA [36]	DSS-MS
Sign	$10G_1^m + 3G_1^a + 2H + 3T_p + 3G_T^p$	$2(4G_1^m + G_1^a + 3H + T_p + G_T^p)$	$8G_1^m + (l+1)G_1^a + 2H + H_p$	$4G_1^m + 2G_1^a + 2H$
Sanitizing	$9G_1^m + 3G_1^a + H + 3T_p + 3G_T^p$	$4G_1^m + G_1^a + 5H + 4T_p + G_T^p$	$3G_1^m + 2lG_1^a + 2H_p + 10T_p$	$G_1^m + G_1^a + H$
Verify	$10G_1^m + 5G_1^a + H + 5T_p + 4G_T^p$	$6G_1^m + G_1^a + 6H + 6T_p + G_T^p$	$4G_1^m + (l+2)G_1^a + 2H_p + 4T_p + 2G_T^p$	$4G_1^m + 3G_1^a + H$

neglecting operations whose costs are negligible, such as large integer arithmetic. For a fair comparison, we instantiate the BFLS10 generic sanitizable signature construction with the widely adopted and efficient BBS group signature, and use the Schnorr signature for signing. Table I summarizes the dominant operations in each scheme across the four phases.

From the table I, it is evident that the proposed DSS-MS only involves point multiplication, point addition, and standard hash functions on elliptic curves, achieving lower cost than the other schemes. In contrast, BFLS10, DVAS, and IBSDIA rely heavily on costly pairing, resulting in complex and resource-intensive computations, which may not be suitable for UAVs with limited energy and processing power.

To further validate our theoretical analysis and assess run-time efficiency, we evaluated the time cost using benchmark. Here, the verification results were measured on the UAV, while the other algorithms were tested on the PC, to better simulate real-world conditions. The results are shown in Fig. 4.

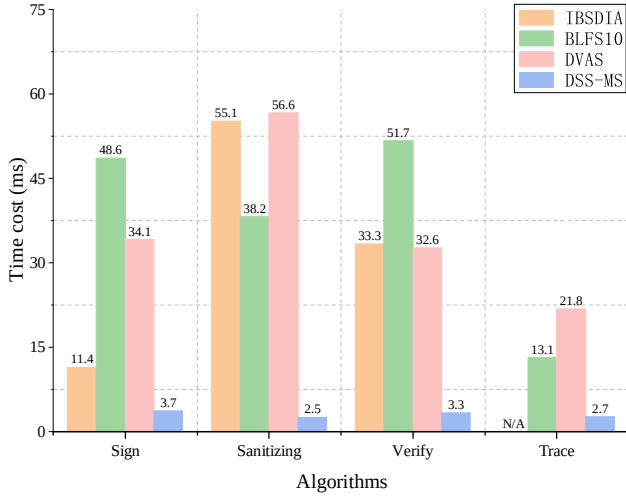


Fig. 4. Time overhead for signing, sanitizing, verification, and tracing. “N/A” indicates that the scheme does not support the tracing functionality.

As illustrated in Fig. 4, the proposed DSS-MS achieves superior efficiency in signing, sanitization, verification, and tracing, with all operations completing in under 4 ms. These results are significantly better than those of the compared schemes, primarily because our design relies only on lightweight elliptic-curve operations and hash functions, whereas most other privacy-preserving approaches depend

heavily on bilinear pairings. Notably, IBSDIA does not support the traceability feature (marked as “N/A”), and therefore lacks accountability functionality.

To further evaluate the performance of our scheme in managing dynamic sanitizers, we measured the time overhead of batch join and revoke operations under different system scales. The results are shown in Fig. 5 and Fig. 6.

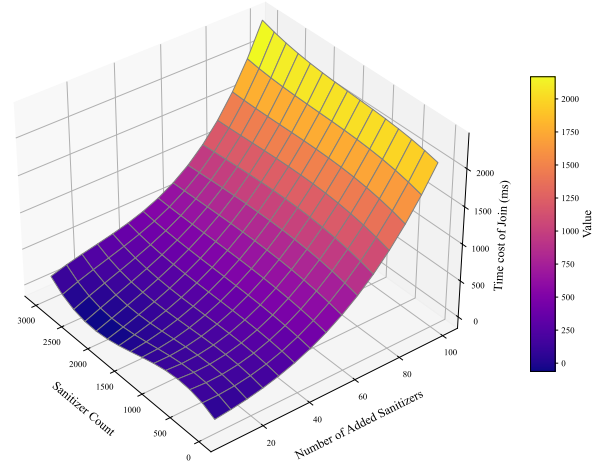


Fig. 5. Join time with varying sanitizer and addition counts.

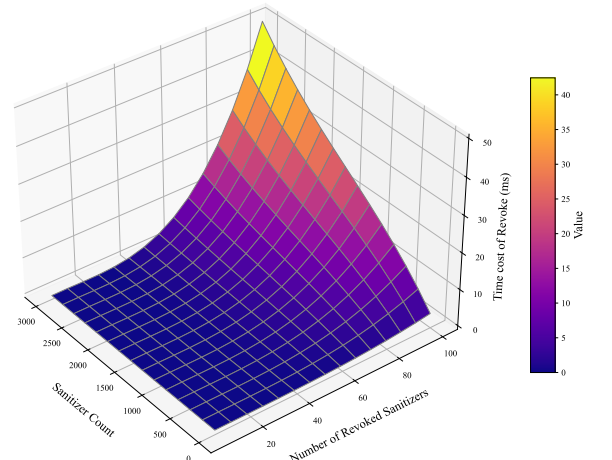


Fig. 6. Revoke time with varying sanitizer and revocation counts.

Fig. 5 presents the time cost of batch joining sanitizers under different system sizes. As shown, the joining time increases significantly with the number of existing sanitizers. It also rises as more sanitizers are added in a single batch. This is because in our Join algorithm, when new sanitizers are introduced, the system needs to recompute parameters such as M_i and y_i based on the CRT, whose computational complexity grows linearly with the total number of sanitizers. Therefore, a larger system scale results in higher joining costs.

Fig. 6 illustrates the time cost of batch revocation operations. When revoking a small number of sanitizers, the revocation time remains nearly constant and largely unaffected by system size. This is because our scheme adopts a lightweight revocation approach that only removes the private key shares of the revoked sanitizers from the aggregated system parameter S_x . Even in an extreme case where 100 sanitizers are revoked at once from a system of 3000 participants, the total time cost remains under 50 ms.

In summary, DSS-MS is a lightweight yet fully functional solution that significantly reduces computational cost, making it well-suited for privacy-preserving UAV logistics with limited resources and real-time requirements.

C. Communication Cost Analysis

In this section, we evaluate and compare the communication overheads of our proposed scheme with three representative existing schemes: BFLS10, DVAS, and IBSDIA. Specifically, we focus on the total size of the signature in each scheme, including all necessary components transmitted during signing. The results are summarized in Table II.

TABLE II
COMPUTATION COST OF FOUR SCHEMES

Scheme	Phase	Theoretical Size	Size at 128-bit Security
BFLS10 [22]		$7Z_p + 4G_1$	532 Bytes
DVAS [32]		$3G_1$	147 Bytes
IBSDIA [36]		$2Z_p + 6G_1$	390 Bytes
Our Scheme		$2Z_p + 2G_1$	194 Bytes

To ensure a fair comparison, we consider the sizes under a 128-bit security level, where an element in Z_p is 48 bytes, and a compressed elliptic curve point in G_1 is 49 bytes. As shown in the table, the BFLS10 scheme incurs the largest communication overhead, with a signature size of 532 bytes. This is mainly due to its reliance on group signature structures, which involve multiple Z_p values and G_1 elements. Similarly, IBSDIA generates a 390 byte signature, resulting from the inclusion of 6 group elements in G_1 and two scalar values in Z_p . The DVAS scheme offers a relatively compact signature size of 147 bytes, consisting of three G_1 elements.

In contrast, our proposed scheme achieves a favorable trade-off between functionality and communication efficiency. With only two Z_p values and two G_1 elements, the total signature size is 194 bytes—much smaller than BFLS10 and IBSDIA, while still providing support for sanitizer flexibility and traceability. This indicates that our scheme significantly reduces communication costs without compromising security or functionality, making it particularly suitable for bandwidth-constrained environments such as UAV networks.

D. Communication Delay

We deployed and executed the order initiator (signer) and the logistics warehouse system (sanitizer) on Ubuntu 24.04.1 running on a PC. For the UAV, we used an Amov P600 equipped with Homer modules to support ad-hoc networking. During the experiment, the UAV and the host machine running the virtual environment were placed in separate rooms. The experimental procedure was as follows: the order initiator generated a signature for the order and sent it to the warehouse node; the warehouse node sanitized the order by removing sensitive information and then forwarded the sanitized order and its signature to the UAV, which verified the validity of the signature.

Experimental results demonstrate that the UAV successfully received and verified the sanitized order with a total end-to-end latency of approximately 183 ms. This delay is primarily attributed to network transmission factors and protocol overheads rather than the execution of cryptographic algorithms. Specifically, the reliance on TCP and WebSocket protocols for reliable connectivity introduced inherent data framing and handshake overheads. Moreover, as the UAV and the host machine were deployed in separate rooms, the intervening walls obstructed the wireless signal. This physical barrier caused signal attenuation which served as the dominant source of the observed latency.

IX. CONCLUSION

We propose a dynamic sanitizable signature scheme with multiple sanitizers (DSS-MS) to address the challenge of protecting user privacy in typical UAV logistics systems. The scheme ensures that logistics orders remain verifiable, sensitive order data is securely protected, and accountability can be enforced when privacy breaches occur. Moreover, the flexible sanitizer management mechanism allows the system to scale efficiently and adapt to the dynamic needs of logistics and warehouse operations. We provide formal security proofs demonstrating that the DSS-MS satisfies a range of security properties, including unforgeability, privacy preservation, among others. To assess its practical performance, we implemented the scheme on a personal computer and the Amov P600 UAV platform. Experimental results show that signature generation, privacy sanitization, verification, and accountability can all be completed within 4 milliseconds, demonstrating high efficiency in both computation and communication. Overall, the proposed DSS-MS scheme offers a practical and reliable privacy-preserving solution for UAV logistics systems and has strong potential for broader use in other privacy-sensitive applications involving multiple participants.

REFERENCES

- [1] M. A. Khan, I. Ullah, A. Alkhalifah, S. U. Rehman, J. A. Shah, M. I. Uddin, M. H. Alsharif, and F. Algarni, "A provable and privacy-preserving authentication scheme for uav-enabled intelligent transportation systems," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 5, pp. 3416–3425, 2022.
- [2] Y. Chen, D. Yang, L. Xiao, F. Wu, and Y. Xu, "Optimal trajectory design for unmanned aerial vehicle cargo pickup and delivery system based on radio map," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 8, pp. 11706–11718, 2024.

- [3] K. Ramana, G. Srivastava, M. R. Kumar, T. R. Gadekallu, J. C.-W. Lin, M. Alazab, and C. Iwendi, "A vision transformer approach for traffic congestion prediction in urban areas," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 4, pp. 3922–3934, 2023.
- [4] F. Hong, G. Wu, Q. Luo, H. Liu, X. Fang, and W. Pedrycz, "Logistics in the sky: A two-phase optimization approach for the drone package pickup and delivery system," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 9, pp. 9175–9190, 2023.
- [5] F. Kong, J. Li, B. Jiang, H. Wang, and H. Song, "Trajectory optimization for drone logistics delivery via attention-based pointer network," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 4, pp. 4519–4531, 2023.
- [6] S. Wandelt, S. Wang, C. Zheng, and X. Sun, "Aerial: A meta review and discussion of challenges toward unmanned aerial vehicle operations in logistics, mobility, and monitoring," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 7, pp. 6276–6289, 2024.
- [7] S. Zhang, H. Zhang, Q. He, K. Bian, and L. Song, "Joint trajectory and power optimization for uav relay networks," *IEEE Communications Letters*, vol. 22, no. 1, pp. 161–164, 2018.
- [8] Y. Zeng, Q. Wu, and R. Zhang, "Accessing from the sky: A tutorial on uav communications for 5g and beyond," *Proceedings of the IEEE*, vol. 107, no. 12, pp. 2327–2375, 2019.
- [9] Y. Zeng, J. Lyu, and R. Zhang, "Cellular-connected uav: Potential, challenges, and promising technologies," *IEEE Wireless Communications*, vol. 26, no. 1, pp. 120–127, 2019.
- [10] "Amazon prime air," Jan. 2025. <https://www.aboutamazon.com/news/transportation/amazon-first-drone-delivery-uk-prime-air-location>.
- [11] H. Song, R. Srinivasan, T. Sookoor, and S. Jeschke, *Smart cities: foundations, principles, and applications*. John Wiley & Sons, 2017.
- [12] S. Iranmanesh, F. S. Abkenar, R. Raad, and A. Jamalipour, "Improving throughput of 5g cellular networks via 3d placement optimization of logistics drones," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 2, pp. 1448–1460, 2021.
- [13] H. Guo, W. Gan, M. Zhao, C. Zhang, T. Wu, L. Zhu, and J. Xue, "Prichain: Efficient privacy-preserving fine-grained redactable blockchains in decentralized settings," *Chinese Journal of Electronics*, vol. 34, no. 1, pp. 82–97, 2025.
- [14] Y. Yao, P. Dash, and K. Pattabiraman, "Poster: May the swarm be with you: Sensor spoofing attacks against drone swarms," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pp. 3511–3513, 2022.
- [15] P. Gope and B. Sikdar, "An efficient privacy-preserving authenticated key agreement scheme for edge-assisted internet of drones," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 11, pp. 13621–13630, 2020.
- [16] J. Sun, G. Xu, H. Li, T. Zhang, C. Wu, X. Yang, and R. H. Deng, "Sanitizable cross-domain access control with policy-driven dynamic authorization," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–17, 2025.
- [17] R. Alnashwan, Y. Yang, Y. Dong, P. Gope, B. Abdolmaleki, and S. R. Hussain, "Strong privacy-preserving universally composable aka protocol with seamless handover support for mobile virtual network operator," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS '24*, (New York, NY, USA), p. 2057–2071, Association for Computing Machinery, 2024.
- [18] R. Alnashwan, P. Gope, and B. Dowling, "Privacy-aware secure region-based handover for small cell networks in 5g-enabled mobile communication," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 1898–1913, 2023.
- [19] F. Wang, J. Cui, Q. Zhang, D. He, and H. Zhong, "Blockchain-assisted flexible revocable anonymous authentication in industrial internet of things," *IEEE Transactions on Network Science and Engineering*, vol. 12, no. 1, pp. 518–532, 2025.
- [20] M. Gojković and M. Schranz, "Preserving privacy in logistics by using swarm intelligence from the bottom-up," in *2024 IEEE 12th International Conference on Intelligent Systems (IS)*, pp. 1–7, 2024.
- [21] G. Ateniese, D. H. Chou, B. de Medeiros, and G. Tsudik, "Sanitizable signatures," in *Computer Security – ESORICS 2005* (S. d. C. di Vimercati, P. Syverson, and D. Gollmann, eds.), (Berlin, Heidelberg), pp. 159–177, Springer Berlin Heidelberg, 2005.
- [22] C. Brzuska, M. Fischlin, A. Lehmann, and D. Schröder, "Unlinkability of sanitizable signatures," in *Public Key Cryptography – PKC 2010* (P. Q. Nguyen and D. Pointcheval, eds.), (Berlin, Heidelberg), pp. 444–461, Springer Berlin Heidelberg, 2010.
- [23] J. Zhang, J. Cui, H. Zhong, Z. Chen, and L. Liu, "Pa-crt: Chinese remainder theorem based conditional privacy-preserving authentication scheme in vehicular ad-hoc networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 2, pp. 722–735, 2021.
- [24] C. Brzuska, M. Fischlin, T. Freudenreich, A. Lehmann, M. Page, J. Schelbert, D. Schröder, and F. Volk, "Security of sanitizable signatures revisited," in *Public Key Cryptography – PKC 2009* (S. Jarecki and G. Tsudik, eds.), (Berlin, Heidelberg), pp. 317–336, Springer Berlin Heidelberg, 2009.
- [25] N. Fleischhacker, J. Krupp, G. Malavolta, J. Schneider, D. Schröder, and M. Simkin, "Efficient unlinkable sanitizable signatures from signatures with re-randomizable keys," in *Public-Key Cryptography – PKC 2016* (C.-M. Cheng, K.-M. Chung, G. Persiano, and B.-Y. Yang, eds.), (Berlin, Heidelberg), pp. 301–330, Springer Berlin Heidelberg, 2016.
- [26] J. Camenisch, D. Derler, S. Krenn, H. C. Pöhls, K. Samelin, and D. Slamanig, "Chameleon-hashes with ephemeral trapdoors," in *Public-Key Cryptography – PKC 2017* (S. Fehr, ed.), (Berlin, Heidelberg), pp. 152–182, Springer Berlin Heidelberg, 2017.
- [27] Y. Zhan, B. Yi, Y. Yang, R. Shi, C. Dong, and M. Huang, "A privilege-constrained sanitizable signature scheme for e-health systems," *Journal of Systems Architecture*, vol. 142, p. 102939, 2023.
- [28] Y. Yang, Q. Shi, P. Gope, B. Abdolmaleki, and B. Sikdar, "Pgus: Pretty good user security for thick mvnos with a novel sanitizable blind signature," in *2025 IEEE Symposium on Security and Privacy (SP)*, pp. 1102–1120, IEEE Computer Society, 2025.
- [29] W. Shen, J. Qin, J. Yu, R. Hao, and J. Hu, "Enabling identity-based integrity auditing and data sharing with sensitive information hiding for secure cloud storage," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 2, pp. 331–346, 2019.
- [30] Z. Liu, L. Ren, R. Li, Q. Liu, and Y. Zhao, "Id-based sanitizable signature data integrity auditing scheme with privacy-preserving," *Computers & Security*, vol. 121, p. 102858, 2022.
- [31] X. Gao, J. Yu, Y. Chang, H. Wang, and J. Fan, "Checking only when it is necessary: Enabling integrity auditing based on the keyword with sensitive information privacy for encrypted cloud data," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 3774–3789, 2022.
- [32] T. Li, H. Wang, D. He, and J. Yu, "Designated-verifier aggregate signature scheme with sensitive data privacy protection for permissioned blockchain-assisted iiot," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 4640–4651, 2023.
- [33] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.
- [34] M. Bellare and G. Neven, "Multi-signatures in the plain public-key model and a general forking lemma," in *Proceedings of the 13th ACM Conference on Computer and Communications Security, CCS '06*, (New York, NY, USA), p. 390–399, Association for Computing Machinery, 2006.
- [35] G. Fuchsbauer, E. Kiltz, and J. Loss, "The algebraic group model and its applications," in *Advances in Cryptology – CRYPTO 2018* (H. Shacham and A. Boldyreva, eds.), (Cham), pp. 33–62, Springer International Publishing, 2018.
- [36] Z. Liu, L. Ren, R. Li, Q. Liu, and Y. Zhao, "Id-based sanitizable signature data integrity auditing scheme with privacy-preserving," *Computers & Security*, vol. 121, p. 102858, 2022.
- [37] "Prometheus/README_en.md at v1.1 · amov-lab/Prometheus," Jan. 2025. <https://github.com/amov-lab/Prometheus>.
- [38] "miracl/core," Jan. 2025. <https://github.com/miracl/core>.
- [39] "google/benchmark," Jan. 2025. <https://github.com/google/benchmark>.

Mingwei Zeng is now a research student in the School of Computer Science and Technology, Anhui University. His research focuses on the security of the Industrial Internet of Things.

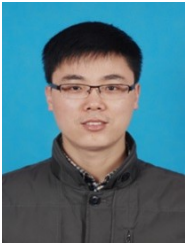




Qingyang Zhang was born in Anhui Province, China, in 1992. He received his B. Eng. degree and Ph.D. degree in computer science from Anhui University in 2021. He is currently an associate professor of School of Computer Science and Technology at Anhui University. His research interest includes edge computing, computer systems, and security. He has over 30 scientific publications in reputable journals (e.g. Proceedings of the IEEE, IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Information Forensics and Security, IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Computers) and international conferences.



Fengqun Wang was born in Anhui Province, China, in 1996. He received his Ph.D. degree in computer science from Anhui University in 2024. He is currently a lecture of School of Computer Science and Technology at Anhui University. His research interests include IoT security, blockchain and applied cryptography. He has multiple scientific publications in reputable journals (e.g. IEEE Transactions on Information Forensics and Security, IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Network and Service Management, IEEE Transactions on Industrial Electronics).



Jie Cui (Senior Member, IEEE) was born in Henan Province, China, in 1980. He received his Ph.D. degree in University of Science and Technology of China in 2012. He is currently a professor and Ph.D. supervisor of the School of Computer Science and Technology at Anhui University. His current research interests include applied cryptography, IoT security, vehicular ad hoc network, cloud computing security and software-defined networking (SDN). He has over 150 scientific publications in reputable journals (e.g. IEEE Transactions on Dependable and

Secure Computing, IEEE Transactions on Information Forensics and Security, IEEE Journal on Selected Areas in Communications, IEEE Transactions on Mobile Computing, IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Computers, IEEE Transactions on Intelligent Transportation Systems, IEEE Transactions on Network and Service Management, IEEE Transactions on Industrial Informatics, IEEE Transactions on Industrial Electronics, IEEE Transactions on Cloud Computing and IEEE Transactions on Multimedia), academic books and international conferences. He is in the Editorial Board of several international journals, such as IET Communications, Security and Communication Networks, and Sensors.



Hong Zhong was born in Anhui Province, China, in 1965. She received her PhD degree in computer science from University of Science and Technology of China in 2005. She is currently a professor and Ph.D. supervisor of the School of Computer Science and Technology at Anhui University. Her research interests include applied cryptography, IoT security, vehicular ad hoc network, cloud computing security and software-defined networking (SDN). She has over 200 scientific publications in reputable journals (e.g. IEEE Journal on Selected Areas in

Communications, IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Mobile Computing, IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Information Forensics and Security, IEEE Transactions on Intelligent Transportation Systems, IEEE Transactions on Multimedia, IEEE Transactions on Vehicular Technology, IEEE Transactions on Network and Service Management, IEEE Transactions on Cloud Computing, IEEE Transactions on Industrial Informatics, IEEE Transactions on Industrial Electronics and IEEE Transactions on Big Data), academic books and international conferences.